

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Курсовой проект
по курсу «Операционные системы»**

Выполнил: А. В. Козлов
Группа: М8О-207БВ-24
Преподаватель: Е. С. Миронов

Москва, 2025

Условие

Цель работы:

1. Приобретение практических навыков в использовании знаний, полученных в течении курса
2. Проведение исследования в выбранной предметной области

Задание: Необходимо спроектировать и реализовать программный прототип в соответствии с выбранным вариантом. Произвести анализ и сделать вывод на основании данных, полученных при работе программного прототипа.

«Быки и коровы» (угадывать необходимо слова). Общение между сервером и клиентом необходимо организовать при помощи метода `map`. При создании каждой игры необходимо указывать количество игроков, которые будут участвовать. То есть угадывать могут несколько игроков. Если кто-то из игроков вышел из игры, то игра должна быть продолжена.

Вариант: 11

Метод решения

Программа реализована как клиент-серверное приложение для игры «Быки и коровы», использующее межпроцессное взаимодействие через разделяемую память и семафоры.

Серверная часть (`server.cpp`) создаёт две области разделяемой памяти — для запросов и ответов — и семафор для синхронизации. В основном цикле сервер ожидает сообщения от клиентов, парсит команды (`CREATE`, `JOIN`, `GUESS`, `LIST`, `LEAVE`, `STATUS`, `QUIT`), выполняет игровую логику (проверка слов, подсчёт быков и коров, управление игровыми сессиями) и отправляет ответ обратно в разделяемую память. Для очистки неактивных игр запускается фоновый поток. Обработчик сигналов обеспечивает корректное завершение работы.

Клиентская часть (`client.cpp`) подключается к тем же ресурсам разделяемой памяти и семафору. В отдельном потоке выводится текстовое меню для взаимодействия с пользователем. При выборе действия клиент формирует сообщение, отправляет его в разделяемую память, уведомляет сервер и ожидает ответа, который затем отображается пользователю.

Абстракция ОС (`os.h`, `os.cpp`) инкапсулирует платформо-зависимые операции (работа с памятью, семафорами, потоками, сигналами) через единый интерфейс `OSInterface`, что позволяет поддерживать разные операционные системы. В текущей реализации используется версия для Linux.

Протокол обмена — текстовый, с разделителем `!`. Семафор гарантирует исключительный доступ к разделам памяти, предотвращая состояния гонки.

Описание программы

Архитектура построена по принципу централизованного сервера, управляющего игровыми сессиями, и клиентов, предоставляющих пользовательский интерфейс. Для обеспечения кросс-платформенной совместимости создан абстрактный слой операционной системы, инкапсулирующий системные вызовы работы с памятью, семафорами и потоками.

Программа использует две области разделяемой памяти: одна для передачи запросов от клиентов к серверу, другая — для ответов сервера клиентам. Синхронизация доступа к памяти осуществляется с помощью двоичного семафора, который предотвращает

одновременную запись и чтение, обеспечивая целостность данных. Сервер запускает фоновый поток для периодической очистки неактивных игр, а также обрабатывает сигналы прерывания для корректного завершения работы.

Клиентская часть предоставляет текстовое меню для взаимодействия с пользователем и работает в отдельном потоке, что позволяет не блокировать основной поток при ожидании ответа от сервера. Поддерживаемые команды включают создание игры, присоединение к существующей, отправку предположения, просмотр списка активных игр, выход из игры и проверку статуса. Все сообщения форматируются в виде строк с разделителями «|» для простоты парсинга.

Программа корректно обрабатывает ошибки подключения к ресурсам разделяемой памяти и семафорам, выводит диагностические сообщения и гарантирует освобождение всех системных ресурсов при завершении работы. Текущая реализация поддерживает Linux, но благодаря абстрактному интерфейсу ОС может быть расширена для других операционных систем.

Результаты

Клиент //

Enter your name: Иван

===== Bulls and Cows Client Player: Иван =====

[CLIENT] Connected to shared memory Connected to server on Linux!

=== Bulls and Cows Game === Player: Иван 1. Create new game 2. Join existing game 3. Make a guess 4. List all games 5. Leave current game 6. Check game status 7. Exit Choose option: 1 Enter game name: ПерваяИгра [CLIENT] Sent request: Иван|CREATE|ПерваяИгра [CLIENT] Received response: OK|Game created|Secret word set Server response: OK Game created Secret word set Game 'ПерваяИгра' created successfully!

=== Bulls and Cows Game === Player: Иван Current game: ПерваяИгра 1. Create new game 2. Join existing game 3. Make a guess 4. List all games 5. Leave current game 6. Check game status 7. Exit Choose option: 3 Enter your 5-letter guess: apple [CLIENT] Sent request: Иван|GUESS|ПерваяИгра|apple [CLIENT] Received response: RESULT|Bulls: 1|Cows: 2 Result: RESULT Bulls: 1 Cows: 2

=== Bulls and Cows Game === Player: Иван Current game: ПерваяИгра 1. Create new game 2. Join existing game 3. Make a guess 4. List all games 5. Leave current game 6. Check game status 7. Exit Choose option: 4 [CLIENT] Sent request: Иван|LIST| [CLIENT] Received response: LIST|ПерваяИгра (1 players) Available games: LIST ПерваяИгра (1 players)

=== Bulls and Cows Game === Player: Иван Current game: ПерваяИгра 1. Create new game 2. Join existing game 3. Make a guess 4. List all games 5. Leave current game 6. Check game status 7. Exit Choose option: 7 Disconnecting from server... [CLIENT] Sent request: Иван|QUIT| Goodbye!

Сервер //

[SERVER] Shared memory initialized with 2 buffers [SERVER] Server started on Linux [SERVER] Waiting for clients... [SERVER] Press Ctrl+C to stop the server.

[SERVER] Received request: Иван|CREATE|ПерваяИгра [SERVER] Processing command: CREATE from Иван [SERVER] Game 'ПерваяИгра' created with secret word: music [SERVER] Sent response: OK|Game created|Secret word set

[SERVER] Received request: Иван|GUESS|ПерваяИгра|apple [SERVER] Processing command:

GUESS from Иван [SERVER] Sent response: RESULT|Bulls: 1|Cows: 2

[SERVER] Received request: Иван|LIST| [SERVER] Processing command: LIST from Иван

[SERVER] Sent response: LIST|ПерваяИгра (1 players)

[SERVER] Received request: Иван|QUIT| [SERVER] Processing command: QUIT from Иван

Выводы

Разработанная программа успешно демонстрирует организацию межпроцессного взаимодействия через разделяемую память в архитектуре «клиент-сервер». Использование двух областей общей памяти (для запросов и ответов) совместно с двоичным семафором обеспечивает синхронизированный и надёжный обмен данными между несколькими клиентами и одним сервером.

Реализация абстрактного OS-слоя (OSInterface) позволила отделить системно-зависимый код от игровой логики и обеспечить переносимость. Поддерживается работа под Linux с использованием POSIX-интерфейсов (shm_open, sem_open, pthread). Программа корректно управляет ресурсами: создаёт и удаляет игры, обрабатывает предположения, очищает неактивные сессии в фоновом потоке и завершает работу по сигналу.

Клиентский интерфейс предоставляет удобное консольное меню с неблокирующим вводом. Все сообщения передаются в простом текстовом формате, что упрощает отладку. Программа устойчиво обрабатывает пользовательский ввод и ошибки соединения.

В целом, проект иллюстрирует практическое применение разделяемой памяти и семафоров для построения многопользовательского приложения с централизованной логикой на сервере.

Исходная программа

```
1  #pragma once
2
3  #include <string>
4  #include <memory>
5
6  namespace OSSignals {
7      const int SIG_INT = 2;
8      const int SIG_TERM = 15;
9  }
10
11
12  using SignalHandler = void(*)(int);
13
14  class OSInterface {
15  public:
16      virtual ~OSInterface() = default;
17
18      virtual bool createSharedMemory(const std::string& name, size_t size) = 0;
19      virtual bool openSharedMemory(const std::string& name) = 0;
20      virtual char* mapSharedMemory(size_t size) = 0;
21      virtual void unmapSharedMemory() = 0;
22      virtual void closeSharedMemory() = 0;
23      virtual void destroySharedMemory(const std::string& name) = 0;
24
25      virtual bool createSemaphore(const std::string& name) = 0;
26      virtual bool openSemaphore(const std::string& name) = 0;
27      virtual void waitSemaphore() = 0;
28      virtual void postSemaphore() = 0;
29      virtual void closeSemaphore() = 0;
30      virtual void destroySemaphore(const std::string& name) = 0;
31
32      virtual void* createThread(void* (*start_routine)(void*), void* arg) = 0;
33      virtual void joinThread(void* thread) = 0;
34      virtual void detachThread(void* thread) = 0;
35
36      virtual void sleepMilliseconds(unsigned int ms) = 0;
37      virtual void sleepMicroseconds(unsigned int us) = 0;
38
39      virtual void setupSignalHandler(int signal, SignalHandler handler) = 0;
40      virtual void setupInterruptHandler(SignalHandler handler) = 0;
41
42      virtual bool fileExists(const std::string& path) = 0;
43
44      virtual std::string getOSName() = 0;
45
46      virtual void cleanupResources() = 0;
47
48  protected:
49      int shm_fd = -1;
50      char* shm_ptr = nullptr;
51      void* semaphore = nullptr;
52  };
53
54  class OSFactory {
55  public:
56      enum OSType {
```

```

57     OS_LINUX,
58     OS_WINDOWS,
59     OS_MACOS
60 };
61
62     static std::unique_ptr<OSInterface> create(OSType type);
63     static std::unique_ptr<OSInterface> createForCurrentOS();
64
65 private:
66     static OSType detectOS();
67 };
68
69 namespace SharedResources {
70     const std::string SHARED_MEMORY_REQUEST = "/bulls_cows_shm_request";
71     const std::string SHARED_MEMORY_RESPONSE = "/bulls_cows_shm_response";
72     const std::string SEMAPHORE_NAME = "/bulls_cows_sem";
73     const size_t DEFAULT_SHM_SIZE = 4096;
74 }

```

Листинг 1: *Интерфейс вызовов*

```

1  #include "os.h"
2  #include <iostream>
3  #include <cstring>
4  #include <cerrno>
5  #include <unistd.h>
6  #include <sys/mman.h>
7  #include <sys/stat.h>
8  #include <sys/types.h>
9  #include <fcntl.h>
10 #include <semaphore.h>
11 #include <pthread.h>
12 #include <signal.h>
13 #include <atomic>
14 #include <cstdlib>
15
16 class OSLinux : public OSInterface {
17 private:
18     pthread_t current_thread;
19     std::atomic<bool> resources_cleaned{false};
20     SignalHandler current_signal_handler = nullptr;
21
22 public:
23     OSLinux() = default;
24     ~OSLinux() override {
25         cleanupResources();
26     }
27
28     bool createSharedMemory(const std::string& name, size_t size) override {
29         shm_fd = shm_open(name.c_str(), O_CREAT | O_RDWR, 0666);
30         if (shm_fd == -1) {
31             std::cerr << "Failed to create shared memory: " << strerror(errno) << std::endl;
32             return false;
33         }
34
35         if (ftruncate(shm_fd, size) == -1) {

```

```

36         std::cerr << "Failed to set shared memory size: " << strerror(errno) << std
37             ::endl;
38         close(shm_fd);
39         return false;
40     }
41     return true;
42 }
43
44 bool openSharedMemory(const std::string& name) override {
45     shm_fd = shm_open(name.c_str(), O_RDWR, 0666);
46     if (shm_fd == -1) {
47         std::cerr << "Failed to open shared memory: " << strerror(errno) << std::
48             endl;
49         return false;
50     }
51     return true;
52 }
53
54 char* mapSharedMemory(size_t size) override {
55     shm_ptr = (char*)mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd,
56         0);
57     if (shm_ptr == MAP_FAILED) {
58         std::cerr << "Failed to map shared memory: " << strerror(errno) << std::
59             endl;
60         shm_ptr = nullptr;
61         return nullptr;
62     }
63     return shm_ptr;
64 }
65
66 void unmapSharedMemory() override {
67     if (shm_ptr != nullptr && shm_ptr != MAP_FAILED) {
68         munmap(shm_ptr, SharedResources::DEFAULT_SHM_SIZE);
69         shm_ptr = nullptr;
70     }
71 }
72
73 void closeSharedMemory() override {
74     if (shm_fd != -1) {
75         close(shm_fd);
76         shm_fd = -1;
77     }
78 }
79
80 void destroySharedMemory(const std::string& name) override {
81     shm_unlink(name.c_str());
82 }
83
84 bool createSemaphore(const std::string& name) override {
85     semaphore = sem_open(name.c_str(), O_CREAT, 0644, 1);
86     if (semaphore == SEM_FAILED) {
87         std::cerr << "Failed to create semaphore: " << strerror(errno) << std::endl
88             ;
89         return false;
90     }
91     return true;
92 }

```

```

89
90     bool openSemaphore(const std::string& name) override {
91         semaphore = sem_open(name.c_str(), 0);
92         if (semaphore == SEM_FAILED) {
93             std::cerr << "Failed to open semaphore: " << strerror(errno) << std::endl;
94             return false;
95         }
96         return true;
97     }
98
99     void waitSemaphore() override {
100         if (semaphore != SEM_FAILED && semaphore != nullptr) {
101             sem_wait((sem_t*)semaphore);
102         }
103     }
104
105     void postSemaphore() override {
106         if (semaphore != SEM_FAILED && semaphore != nullptr) {
107             sem_post((sem_t*)semaphore);
108         }
109     }
110
111     void closeSemaphore() override {
112         if (semaphore != SEM_FAILED && semaphore != nullptr) {
113             sem_close((sem_t*)semaphore);
114             semaphore = nullptr;
115         }
116     }
117
118     void destroySemaphore(const std::string& name) override {
119         sem_unlink(name.c_str());
120     }
121
122     void* createThread(void* (*start_routine)(void*), void* arg) override {
123         pthread_t* thread = new pthread_t;
124         if (pthread_create(thread, nullptr, start_routine, arg) != 0) {
125             delete thread;
126             return nullptr;
127         }
128         return thread;
129     }
130
131     void joinThread(void* thread) override {
132         if (thread != nullptr) {
133             pthread_join(*static_cast<pthread_t*>(thread), nullptr);
134             delete static_cast<pthread_t*>(thread);
135         }
136     }
137
138     void detachThread(void* thread) override {
139         if (thread != nullptr) {
140             pthread_detach(*static_cast<pthread_t*>(thread));
141         }
142     }
143
144     void sleepMilliseconds(unsigned int ms) override {
145         usleep(ms * 1000);
146     }

```



```

147
148 void sleepMicroseconds(unsigned int us) override {
149     usleep(us);
150 }
151
152 static void signalHandlerWrapper(int sig) {
153     //     sigaction
154     //     setupSignalHandler
155 }
156
157 void setupSignalHandler(int signal, SignalHandler handler) override {
158     current_signal_handler = handler;
159
160     struct sigaction sa;
161     sa.sa_handler = handler;
162     sigemptyset(&sa.sa_mask);
163     sa.sa_flags = 0;
164
165     if (sigaction(signal, &sa, NULL) == -1) {
166         std::cerr << "Failed to setup signal handler: " << strerror(errno) << std:::
            endl;
167     }
168 }
169
170 void setupInterruptHandler(SignalHandler handler) override {
171     setupSignalHandler(OSSignals::SIG_INT, handler);
172 }
173
174 bool fileExists(const std::string& path) override {
175     return access(path.c_str(), F_OK) != -1;
176 }
177
178 std::string getOSName() override {
179     return "Linux";
180 }
181
182 void cleanupResources() override {
183     if (resources_cleaned.exchange(true)) {
184         return;
185     }
186
187     unmapSharedMemory();
188     closeSharedMemory();
189     closeSemaphore();
190 }
191 };
192
193 std::unique_ptr<OSInterface> OSFactory::create(OSType type) {
194     switch (type) {
195         case OS_LINUX:
196             return std::make_unique<OSLinux>();
197         case OS_WINDOWS:
198         case OS_MACOS:
199         default:
200             std::cerr << "OS type not supported yet" << std::endl;
201             return nullptr;
202     }
203 }

```

```

204
205 std::unique_ptr<OSInterface> OSFactory::createForCurrentOS() {
206     OSType type = detectOS();
207     return create(type);
208 }
209
210 OSFactory::OSType OSFactory::detectOS() {
211     return OS_LINUX;
212 }

```

Листинг 2: *Реализация вызовов*

```

1  #include <iostream>
2  #include <fstream>
3  #include <string>
4  #include <vector>
5  #include <map>
6  #include <set>
7  #include <algorithm>
8  #include <cstring>
9  #include <random>
10 #include <ctime>
11 #include <atomic>
12 #include <memory>
13
14 #include "os.h"
15
16 struct Game {
17     std::string name;
18     std::string secretWord;
19     std::set<std::string> players;
20     std::map<std::string, std::string> guesses;
21     bool gameOver;
22     std::string winner;
23 };
24
25 struct ThreadArgs {
26     std::atomic<bool>* running;
27     std::map<std::string, Game>* games;
28     OSInterface* os;
29 };
30
31 class BullsAndCowsServer {
32 private:
33     std::map<std::string, Game> games;
34     std::vector<std::string> wordList;
35     std::unique_ptr<OSInterface> os;
36     std::atomic<bool> running;
37     void* cleanup_thread;
38     char* shm_request_ptr;
39     char* shm_response_ptr;
40
41     static BullsAndCowsServer* server_instance;
42
43 public:
44     BullsAndCowsServer() : running(true), cleanup_thread(nullptr),
45         shm_request_ptr(nullptr), shm_response_ptr(nullptr) {

```

```

46     loadWordList();
47     os = OSFactory::createForCurrentOS();
48     if (!os) {
49         throw std::runtime_error("Failed to create OS interface");
50     }
51     server_instance = this;
52 }
53
54 ~BullsAndCowsServer() {
55     stop();
56     server_instance = nullptr;
57 }
58
59 void loadWordList() {
60     wordList = {
61         "apple", "bread", "chair", "dance", "earth",
62         "flame", "glass", "heart", "image", "jolly",
63         "knife", "lemon", "music", "night", "ocean",
64         "piano", "queen", "river", "smile", "table",
65         "umbra", "vital", "water", "xenon", "yacht",
66         "zebra", "brain", "cloud", "dream", "eagle"
67     };
68 }
69
70 std::string getRandomWord() {
71     std::mt19937 rng(std::time(nullptr));
72     std::uniform_int_distribution<size_t> dist(0, wordList.size() - 1);
73     return wordList[dist(rng)];
74 }
75
76 static void* cleanupGamesThread(void* arg) {
77     ThreadArgs* args = static_cast<ThreadArgs*>(arg);
78
79     while (*(args->running)) {
80         args->os->sleepMilliseconds(60000);
81
82         auto it = args->games->begin();
83         while (it != args->games->end()) {
84             if (it->second.players.empty()) {
85                 std::cout << "[SERVER] Cleaning up empty game: " << it->first << std
86                     ::endl;
87                 it = args->games->erase(it);
88             } else {
89                 ++it;
90             }
91         }
92
93         delete args;
94         return nullptr;
95     }
96
97     bool initialize() {
98         if (!os->createSharedMemory(SharedResources::SHARED_MEMORY_REQUEST,
99                                     SharedResources::DEFAULT_SHM_SIZE)) {
100             return false;
101         }
102

```

```

103     shm_request_ptr = os->mapSharedMemory(SharedResources::DEFAULT_SHM_SIZE);
104     if (!shm_request_ptr) {
105         return false;
106     }
107
108     if (!os->createSharedMemory(SharedResources::SHARED_MEMORY_RESPONSE,
109                               SharedResources::DEFAULT_SHM_SIZE)) {
110         return false;
111     }
112
113     shm_response_ptr = os->mapSharedMemory(SharedResources::DEFAULT_SHM_SIZE);
114     if (!shm_response_ptr) {
115         return false;
116     }
117
118     memset(shm_request_ptr, 0, SharedResources::DEFAULT_SHM_SIZE);
119     memset(shm_response_ptr, 0, SharedResources::DEFAULT_SHM_SIZE);
120
121     if (!os->createSemaphore(SharedResources::SEMAPHORE_NAME)) {
122         return false;
123     }
124
125     ThreadArgs* args = new ThreadArgs(&running, &games, os.get());
126     cleanup_thread = os->createThread(cleanupGamesThread, args);
127     if (!cleanup_thread) {
128         delete args;
129         return false;
130     }
131
132     os->detachThread(cleanup_thread);
133
134     std::cout << "[SERVER] Shared memory initialized with 2 buffers" << std::endl;
135     return true;
136 }
137
138 std::pair<int, int> calculateBullsAndCows(const std::string& secret, const std:::
139     string& guess) {
140     int bulls = 0;
141     int cows = 0;
142
143     for (size_t i = 0; i < secret.length(); ++i) {
144         if (secret[i] == guess[i]) {
145             bulls++;
146         } else if (secret.find(guess[i]) != std::string::npos) {
147             cows++;
148         }
149     }
150
151     return {bulls, cows};
152 }
153
154 void processClientMessage(const std::string& message, std::string& response) {
155     size_t pos1 = message.find('|');
156     size_t pos2 = message.find('|', pos1 + 1);
157
158     if (pos1 == std::string::npos || pos2 == std::string::npos) {
159         response = "ERROR|Invalid message format";
160         return;

```

```

160     }
161
162     std::string playerName = message.substr(0, pos1);
163     std::string command = message.substr(pos1 + 1, pos2 - pos1 - 1);
164     std::string data_msg = message.substr(pos2 + 1);
165
166     std::cout << "[SERVER] Processing command: " << command << " from " <<
        playerName << std::endl;
167
168     if (command == "CREATE") {
169         createGame(playerName, data_msg, response);
170     } else if (command == "JOIN") {
171         joinGame(playerName, data_msg, response);
172     } else if (command == "GUESS") {
173         processGuess(playerName, data_msg, response);
174     } else if (command == "LIST") {
175         listGames(response);
176     } else if (command == "LEAVE") {
177         leaveGame(playerName, data_msg, response);
178     } else if (command == "STATUS") {
179         getGameStatus(data_msg, response);
180     } else if (command == "QUIT") {
181         removePlayerFromAllGames(playerName);
182         response = "OK|Goodbye";
183     } else {
184         response = "ERROR|Unknown command: " + command;
185     }
186 }
187
188 void createGame(const std::string& playerName, const std::string& gameName, std::
    string& response) {
189     if (games.find(gameName) != games.end()) {
190         response = "ERROR|Game already exists";
191         return;
192     }
193
194     Game newGame;
195     newGame.name = gameName;
196     newGame.secretWord = getRandomWord();
197     newGame.players.insert(playerName);
198     newGame.gameOver = false;
199
200     games[gameName] = newGame;
201     std::cout << "[SERVER] Game '" << gameName << "' created with secret word: " <<
        newGame.secretWord << std::endl;
202     response = "OK|Game created|Secret word set";
203 }
204
205 void joinGame(const std::string& playerName, const std::string& gameName, std::
    string& response) {
206     auto it = games.find(gameName);
207     if (it == games.end()) {
208         response = "ERROR|Game not found";
209         return;
210     }
211
212     if (it->second.gameOver) {
213         response = "ERROR|Game is over";

```

```

214         return;
215     }
216
217     it->second.players.insert(playerName);
218     response = "OK|Joined game|Players: " + std::to_string(it->second.players.size
        ());
219 }
220
221 void processGuess(const std::string& playerName, const std::string& data_msg, std
    ::string& response) {
222     size_t pos = data_msg.find('|');
223     if (pos == std::string::npos) {
224         response = "ERROR|Invalid guess format";
225         return;
226     }
227
228     std::string gameName = data_msg.substr(0, pos);
229     std::string guess = data_msg.substr(pos + 1);
230
231     auto it = games.find(gameName);
232     if (it == games.end()) {
233         response = "ERROR|Game not found";
234         return;
235     }
236
237     if (it->second.gameOver) {
238         response = "ERROR|Game is over";
239         return;
240     }
241
242     if (it->second.players.find(playerName) == it->second.players.end()) {
243         response = "ERROR|You are not in this game";
244         return;
245     }
246
247     if (guess.length() != 5) {
248         response = "ERROR|Word must be 5 letters";
249         return;
250     }
251
252     for (char& c : guess) {
253         c = std::tolower(c);
254     }
255
256     it->second.guesses[playerName] = guess;
257
258     if (guess == it->second.secretWord) {
259         it->second.gameOver = true;
260         it->second.winner = playerName;
261         response = "WINNER|" + playerName + "|You guessed the word!";
262     } else {
263         auto [bulls, cows] = calculateBullsAndCows(it->second.secretWord, guess);
264         response = "RESULT|Bulls: " + std::to_string(bulls) + "|Cows: " + std::
            to_string(cows);
265     }
266 }
267
268 void listGames(std::string& response) {

```

```

269     if (games.empty()) {
270         response = "INFO|No active games";
271         return;
272     }
273
274     response = "LIST";
275     for (const auto& [name, game] : games) {
276         response += "|" + name + " (" + std::to_string(game.players.size()) +
277             " players)" + (game.gameOver ? " [Finished]" : "");
278     }
279 }
280
281 void leaveGame(const std::string& playerName, const std::string& gameName, std:::
    string& response) {
282     auto it = games.find(gameName);
283     if (it == games.end()) {
284         response = "ERROR|Game not found";
285         return;
286     }
287
288     it->second.players.erase(playerName);
289     it->second.guesses.erase(playerName);
290
291     if (it->second.players.empty()) {
292         games.erase(it);
293         response = "OK|Game removed";
294     } else {
295         response = "OK|Left game|Remaining players: " + std::to_string(it->second.
            players.size());
296     }
297 }
298
299 void getGameStatus(const std::string& gameName, std::string& response) {
300     auto it = games.find(gameName);
301     if (it == games.end()) {
302         response = "ERROR|Game not found";
303         return;
304     }
305
306     response = "STATUS|Players: " + std::to_string(it->second.players.size()) +
307         "|Game over: " + std::string(it->second.gameOver ? "Yes" : "No");
308
309     if (it->second.gameOver) {
310         response += "|Winner: " + it->second.winner;
311     }
312 }
313
314 void removePlayerFromAllGames(const std::string& playerName) {
315     std::vector<std::string> gamesToRemove;
316
317     for (auto& [gameName, game] : games) {
318         game.players.erase(playerName);
319         game.guesses.erase(playerName);
320
321         if (game.players.empty()) {
322             gamesToRemove.push_back(gameName);
323         }
324     }

```

```

325
326     for (const auto& gameName : gamesToRemove) {
327         games.erase(gameName);
328     }
329 }
330
331 static void signalHandler(int sig) {
332     if (server_instance) {
333         server_instance->handleSignal(sig);
334     }
335 }
336
337 void handleSignal(int sig) {
338     std::cout << "\n[SERVER] Received signal " << sig << ", shutting down..." <<
        std::endl;
339     running = false;
340 }
341
342 void run() {
343     if (!initialize()) {
344         std::cerr << "Failed to initialize server" << std::endl;
345         return;
346     }
347
348     std::cout << "[SERVER] Server started on " << os->getOSName() << std::endl;
349     std::cout << "[SERVER] Waiting for clients..." << std::endl;
350     std::cout << "[SERVER] Press Ctrl+C to stop the server." << std::endl;
351
352     os->setupInterruptHandler(signalHandler);
353
354     while (running) {
355         os->waitSemaphore();
356
357         if (shm_request_ptr && strlen(shm_request_ptr) > 0) {
358             std::string request(shm_request_ptr);
359             std::cout << "\n[SERVER] Received request: " << request << std::endl;
360
361             std::string response;
362             processClientMessage(request, response);
363
364             memset(shm_response_ptr, 0, SharedResources::DEFAULT_SHM_SIZE);
365             strncpy(shm_response_ptr, response.c_str(), SharedResources::
                DEFAULT_SHM_SIZE - 1);
366
367             memset(shm_request_ptr, 0, SharedResources::DEFAULT_SHM_SIZE);
368
369             std::cout << "[SERVER] Sent response: " << response << std::endl;
370         }
371
372         os->postSemaphore();
373
374         os->sleepMilliseconds(50);
375     }
376
377     std::cout << "\n[SERVER] Server stopped." << std::endl;
378 }
379
380 void stop() {

```



```

381         running = false;
382
383         cleanup_thread = nullptr;
384
385         if (shm_request_ptr) {
386             os->unmapSharedMemory();
387             shm_request_ptr = nullptr;
388         }
389         if (shm_response_ptr) {
390             os->unmapSharedMemory();
391             shm_response_ptr = nullptr;
392         }
393         os->destroySharedMemory(SharedResources::SHARED_MEMORY_REQUEST);
394         os->destroySharedMemory(SharedResources::SHARED_MEMORY_RESPONSE);
395         os->destroySemaphore(SharedResources::SEMAPHORE_NAME);
396         os->cleanupResources();
397     }
398 };
399
400 BullsAndCowsServer* BullsAndCowsServer::server_instance = nullptr;
401
402 int main() {
403     try {
404         BullsAndCowsServer server;
405         server.run();
406     } catch (const std::exception& e) {
407         std::cerr << "Error: " << e.what() << std::endl;
408         return 1;
409     }
410
411     return 0;
412 }

```

Листинг 3: *Сервер*

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <cstring>
5  #include <atomic>
6  #include <memory>
7  #include <algorithm>
8  #include <cctype>
9
10 #include "os.h"
11
12 class BullsAndCowsClient;
13
14 struct ClientThreadArgs {
15     std::atomic<bool>* running;
16     BullsAndCowsClient* client;
17 };
18
19 class BullsAndCowsClient {
20 private:
21     std::unique_ptr<OSInterface> os;
22     std::string playerName;

```

```

23     std::string currentGame;
24     std::atomic<bool> running;
25     void* input_thread;
26     char* shm_request_ptr;
27     char* shm_response_ptr;
28
29 public:
30     BullsAndCowsClient(const std::string& name) :
31         playerName(name), running(true), input_thread(nullptr),
32         shm_request_ptr(nullptr), shm_response_ptr(nullptr) {
33         os = OSFactory::createForCurrentOS();
34         if (!os) {
35             throw std::runtime_error("Failed to create OS interface");
36         }
37     }
38
39     ~BullsAndCowsClient() {
40         disconnect();
41     }
42
43     static void* inputHandlerThread(void* arg) {
44         ClientThreadArgs* args = static_cast<ClientThreadArgs*>(arg);
45         args->client->showMenuLoop();
46         delete args;
47         return nullptr;
48     }
49
50     void showMenuLoop() {
51         int choice;
52
53         while (running) {
54             showMenu();
55
56             if (!(std::cin >> choice)) {
57                 std::cin.clear();
58                 std::cin.ignore(10000, '\n');
59                 continue;
60             }
61
62             processChoice(choice);
63         }
64     }
65
66     void showMenu() {
67         std::cout << "\n=== Bulls and Cows Game ===" << std::endl;
68         std::cout << "Player: " << playerName << std::endl;
69         if (!currentGame.empty()) {
70             std::cout << "Current game: " << currentGame << std::endl;
71         }
72         std::cout << "1. Create new game" << std::endl;
73         std::cout << "2. Join existing game" << std::endl;
74         std::cout << "3. Make a guess" << std::endl;
75         std::cout << "4. List all games" << std::endl;
76         std::cout << "5. Leave current game" << std::endl;
77         std::cout << "6. Check game status" << std::endl;
78         std::cout << "7. Exit" << std::endl;
79         std::cout << "Choose option: ";
80     }

```

```

81
82 void processChoice(int choice) {
83     switch (choice) {
84         case 1:
85             createGame();
86             break;
87         case 2:
88             joinGame();
89             break;
90         case 3:
91             makeGuess();
92             break;
93         case 4:
94             listGames();
95             break;
96         case 5:
97             leaveGame();
98             break;
99         case 6:
100             gameStatus();
101             break;
102         case 7:
103             quit();
104             running = false;
105             break;
106         default:
107             std::cout << "Invalid option!" << std::endl;
108     }
109 }
110
111 bool connect() {
112     if (!os->openSharedMemory(SharedResources::SHARED_MEMORY_REQUEST)) {
113         std::cerr << "[CLIENT] Failed to open request shared memory" << std::endl;
114         return false;
115     }
116
117     shm_request_ptr = os->mapSharedMemory(SharedResources::DEFAULT_SHM_SIZE);
118     if (!shm_request_ptr) {
119         std::cerr << "[CLIENT] Failed to map request shared memory" << std::endl;
120         return false;
121     }
122
123     if (!os->openSharedMemory(SharedResources::SHARED_MEMORY_RESPONSE)) {
124         std::cerr << "[CLIENT] Failed to open response shared memory" << std::endl;
125         return false;
126     }
127
128     shm_response_ptr = os->mapSharedMemory(SharedResources::DEFAULT_SHM_SIZE);
129     if (!shm_response_ptr) {
130         std::cerr << "[CLIENT] Failed to map response shared memory" << std::endl;
131         return false;
132     }
133
134     if (!os->openSemaphore(SharedResources::SEMAPHORE_NAME)) {
135         std::cerr << "[CLIENT] Failed to open semaphore" << std::endl;
136         return false;
137     }
138 }

```

```

139     memset(shm_request_ptr, 0, SharedResources::DEFAULT_SHM_SIZE);
140     memset(shm_response_ptr, 0, SharedResources::DEFAULT_SHM_SIZE);
141
142     std::cout << "[CLIENT] Connected to shared memory" << std::endl;
143     return true;
144 }
145
146 std::string sendAndReceive(const std::string& message) {
147     os->waitSemaphore();
148
149     memset(shm_response_ptr, 0, SharedResources::DEFAULT_SHM_SIZE);
150
151     memset(shm_request_ptr, 0, SharedResources::DEFAULT_SHM_SIZE);
152     strncpy(shm_request_ptr, message.c_str(), SharedResources::DEFAULT_SHM_SIZE -
153             1);
154     std::cout << "[CLIENT] Sent request: " << message << std::endl;
155
156     os->postSemaphore();
157
158     int attempts = 0;
159     const int max_attempts = 100;
160     std::string response;
161
162     while (attempts < max_attempts && running) {
163         os->sleepMilliseconds(100);
164
165         os->waitSemaphore();
166
167         if (strlen(shm_response_ptr) > 0) {
168             response = std::string(shm_response_ptr);
169             std::cout << "[CLIENT] Received response: " << response << std::endl;
170
171             memset(shm_request_ptr, 0, SharedResources::DEFAULT_SHM_SIZE);
172
173             os->postSemaphore();
174             break;
175         }
176
177         os->postSemaphore();
178         attempts++;
179     }
180
181     if (attempts >= max_attempts) {
182         std::cerr << "[CLIENT] Timeout waiting for server response" << std::endl;
183         return "ERROR|Server timeout";
184     }
185
186     if (response.empty()) {
187         return "ERROR|Empty response from server";
188     }
189
190     return response;
191 }
192
193 void createGame() {
194     std::string gameName;
195     std::cout << "Enter game name: ";
196     std::cin >> gameName;

```

```

196
197     if (gameName.empty()) {
198         std::cout << "Game name cannot be empty!" << std::endl;
199         return;
200     }
201
202     std::string message = playerName + "|CREATE|" + gameName;
203     std::string response = sendAndReceive(message);
204
205     std::cout << "Server response:\n" << parseResponse(response) << std::endl;
206
207     if (response.find("OK") == 0) {
208         currentGame = gameName;
209         std::cout << " Game '" << gameName << "' created successfully!" << std::
                endl;
210     }
211 }
212
213 void joinGame() {
214     std::string gameName;
215     std::cout << "Enter game name to join: ";
216     std::cin >> gameName;
217
218     if (gameName.empty()) {
219         std::cout << "Game name cannot be empty!" << std::endl;
220         return;
221     }
222
223     std::string message = playerName + "|JOIN|" + gameName;
224     std::string response = sendAndReceive(message);
225
226     std::cout << "Server response:\n" << parseResponse(response) << std::endl;
227
228     if (response.find("OK") == 0) {
229         currentGame = gameName;
230         std::cout << " Joined game '" << gameName << "' successfully!" << std::endl
                ;
231     }
232 }
233
234 void makeGuess() {
235     if (currentGame.empty()) {
236         std::cout << "You are not in any game!" << std::endl;
237         return;
238     }
239
240     std::string guess;
241     std::cout << "Enter your 5-letter guess: ";
242     std::cin >> guess;
243
244     if (guess.length() != 5) {
245         std::cout << "Word must be 5 letters long!" << std::endl;
246         return;
247     }
248
249     for (char& c : guess) {
250         c = std::tolower(c);
251     }

```

```

252
253     std::string message = playerName + "|GUESS|" + currentGame + "|" + guess;
254     std::string response = sendAndReceive(message);
255
256     std::string result = parseResponse(response);
257     std::cout << "Result:\n" << result << std::endl;
258
259     if (response.find("WINNER") == 0) {
260         currentGame.clear();
261         std::cout << " Congratulations! You won the game!" << std::endl;
262     }
263 }
264
265 void listGames() {
266     std::string message = playerName + "|LIST|";
267     std::string response = sendAndReceive(message);
268
269     std::cout << "Available games:\n" << parseResponse(response) << std::endl;
270 }
271
272 void leaveGame() {
273     if (currentGame.empty()) {
274         std::cout << "You are not in any game!" << std::endl;
275         return;
276     }
277
278     std::cout << "Leaving game '" << currentGame << "'..." << std::endl;
279     std::string message = playerName + "|LEAVE|" + currentGame;
280
281     std::string response = sendAndReceive(message);
282
283     std::cout << parseResponse(response) << std::endl;
284     currentGame.clear();
285 }
286
287 void gameStatus() {
288     if (currentGame.empty()) {
289         std::cout << "Enter game name: ";
290         std::string gameName;
291         std::cin >> gameName;
292         currentGame = gameName;
293     }
294
295     std::string message = playerName + "|STATUS|" + currentGame;
296     std::cout << "Checking status of game '" << currentGame << "'..." << std::endl;
297
298     std::string response = sendAndReceive(message);
299
300     std::cout << "Game status:\n" << parseResponse(response) << std::endl;
301 }
302
303 void quit() {
304     std::cout << "Disconnecting from server..." << std::endl;
305
306     if (!currentGame.empty()) {
307         std::string message = playerName + "|LEAVE|" + currentGame;
308         sendAndReceive(message);
309     }

```

```

310
311     std::string message = playerName + "|QUIT|";
312     sendAndReceive(message);
313 }
314
315 std::string parseResponse(const std::string& response) {
316     std::string result;
317     size_t start = 0;
318     size_t end = response.find('|');
319
320     while (end != std::string::npos) {
321         if (start != 0) result += "\n";
322         result += response.substr(start, end - start);
323         start = end + 1;
324         end = response.find('|', start);
325     }
326
327     if (start < response.length()) {
328         if (start != 0) result += "\n";
329         result += response.substr(start);
330     }
331
332     return result;
333 }
334
335 void run() {
336     std::cout << "=====" << std::endl;
337     std::cout << " Bulls and Cows Client" << std::endl;
338     std::cout << " Player: " << playerName << std::endl;
339     std::cout << "=====" << std::endl;
340
341     if (!connect()) {
342         std::cerr << "Failed to connect to server" << std::endl;
343         std::cerr << "Make sure server is running!" << std::endl;
344         return;
345     }
346
347     std::cout << "Connected to server on " << os->getOSName() << "!" << std::endl;
348
349     ClientThreadArgs* args = new ClientThreadArgs();
350     args->running = &running;
351     args->client = this;
352     input_thread = os->createThread(inputHandlerThread, args);
353     if (!input_thread) {
354         delete args;
355         std::cerr << "Failed to create input thread" << std::endl;
356         return;
357     }
358
359     os->joinThread(input_thread);
360     input_thread = nullptr;
361
362     std::cout << "\nGoodbye!" << std::endl;
363 }
364
365 void disconnect() {
366     running = false;
367

```

```

368         if (input_thread) {
369             os->joinThread(input_thread);
370             input_thread = nullptr;
371         }
372
373         os->cleanupResources();
374     }
375 };
376
377 int main() {
378     try {
379         std::string playerName;
380         std::cout << "Enter your name: ";
381         std::getline(std::cin, playerName);
382
383         if (playerName.empty()) {
384             std::cout << "Name cannot be empty. Using default name 'Player'" << std::endl;
385             playerName = "Player";
386         }
387
388         BullsAndCowsClient client(playerName);
389         client.run();
390     } catch (const std::exception& e) {
391         std::cerr << "Error: " << e.what() << std::endl;
392         return 1;
393     }
394
395     return 0;
396 }

```

Листинг 4: *Клиент*